Design of NA Project

WangHao 3220103613 *

(Information and Computing Sciences, Class 2201), Zhejiang University

Due time: December 23, 2024

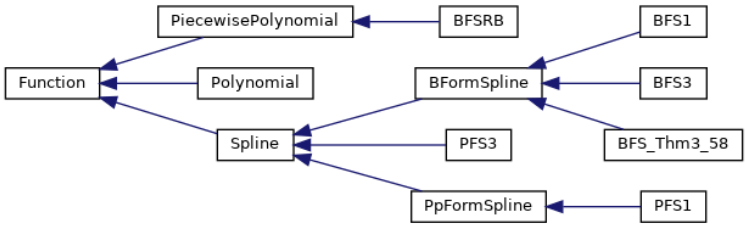
Code reuses

I reuses many files in Programming Hw 2. All of them are listed below.

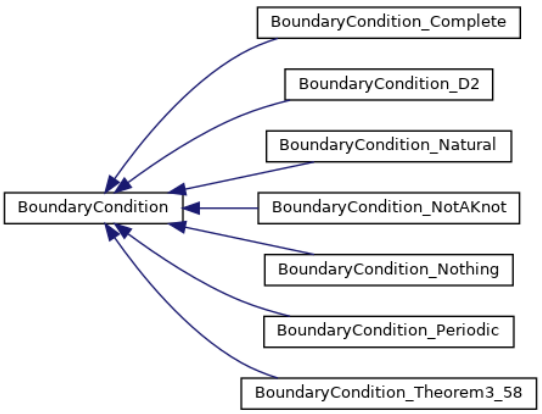
- Exceptions.hpp.
- DS&Constants.cpp(hpp).
- Function.cpp(hpp).
- Polynomial.cpp(hpp).
- EquationSolver.cpp(hpp).
- Curve.cpp(hpp).
- LatexOutputer.cpp(hpp).
- NameLibrary.cpp(hpp).

Class introduction

Below is the inherit graph of my program, including classes that have inheritance relationship.



(a) inherit graph of splines



(b) inherit graph of boundary conditions

Figure 1: inherit graph

All structs and classes are introduced below (including those not appeared in the graph above).

DefinitionDomain It represents definition domain, which can be applied to a function, curve, etc.

*Electronic address: 3220103613@zju.edu.cn

Function It represents function.

On the basis of `class function` in chapter 1, I add a check of definition domain when get value or derivative value.

Curve It represents curve.

Implemented by using `vector<const Function*>`. Every item of the vector is the function of the curve in a certain direction.

Structs that define the points, values, etc. of function and curve There are about ten types, all defined and annotated using Doxygen comment format in file `DS&Constants.hpp`.

Polynomial It represents polynomial.

Derived from `Function`.
Implemented by using `vector<double>` to record the coefficient of x^i.
Allows a lot of operations.

PiecewisePolynomial It represents piecewise polynomial, i.e. a piecewise function and every piece of it is a polynomial.

Derived from `Function`.
Implemented by using `vector<Polynomial>`. As every polynomial in it has disjoint definition domain, it is well-defined. `operator()` is implemented through binary search.

Spline It represents spline $\in \mathbb{S}_n^k, \forall n, k$.

Derived from `Function`.
An abstract class and base class of every specified spline.
Have member variable `PiecewisePolynomial piecewisePolynomial`, recording the spline.
We can notice that generation of spline follows the same pattern:

- get necessary conditions.
- generate matrix A.
- generate vector b.
- add boundary conditions. (complete generation of A and b.)
- solve Ax=b.
- generate spline by x.

Thus I write a function `void generate();`, including steps above. For steps that are different in different kind of splines, I use virtual functions. They are listed below.

- `virtual void generate_A(...);`
- `virtual Eigen::VectorXd generate_b(...);`
- `virtual void addBoundaryCondition(...);`
- `virtual void generate_PiecePoly(...);`

Apparently, those functions are all pure virtual functions in this base class.
Then 2 major kind of splines are derived from this base class.

BoundaryCondition We introduce this before specified splines.

It represents boundary conditions for generation of the spline.
In the base class, I define a `enum class Type`. The base class has a member variable `Type type`, recording the type of the boundary condition. And the constructor of the base class is declared protected to avoid being constructed by users, but allowed to be called by derived class.
Then it derives subclasses, representing different sort of boundary conditions. I defined 7 sort of boundary conditions.

PpFormSpline It represents PP-form spline $\in \mathbb{S}_n^{n-1}, \forall n$.

Derived from `Spline`.
Implemented by override pure virtual functions of the base class. **Related mathematical theories are introduced below.**
pp-form means every piece of the spline are expressed in the form of polynomial. Denote the number of the knots are N, then there are $(N-1)(n+1)$ variables.
Using the continuity of the spline up to the $n-1$th order, we obtain $(n-1)(N-2)$ equations. Using the values

of the spline at the N nodes, we obtain $2 * (N-1)$ equations. Using the boundary conditions, we obtain $n-1$ equations.
Write all of the conditions into the linear system, we get A and b. Then generation of the piecewise polynomial is done by just connect all polynomials (formed by using x).

PFS1 It represents PP-form spline $\in \mathbb{S}_1^0$.
Derived form `PpFormSpline`.
This is the specified case of PpFormSpline, n=1. Thus it is implemented only by call the constructor of the PpFormSpline and set the parameter to be 1.
Overload `void generate();`, as the boundary condition is always nothing.

PFS3 It represents PP-form spline $\in \mathbb{S}_3^2$.
Derived form `Spline`.
I can implement it just like `PFS1`. However, lemmas[1] in book reduces number of variables from $4 * (N-1)$ to N, which means it can be calculated faster. And whole procedure of generating spline is totally different from generating PpFormSpline, given parameter of $n = 3$.
Thus, I derived this class from spline and implement it by using lemmas in book.
It allows 5 kind of boundary conditions (all defined in book[2]), done by override `virtual void addBoundaryCondition(..`
By the way, this is the fastest way in my project to generate a spline $\in \mathbb{S}_3^2$. Thus `class CurveFitter` uses it.

BFSRB It represents B-form spline recursion base, which is defined in book[3].
Derived form `PiecewisePolynomial`. It is constructed by calling the constructor of PiecewisePolynomial, giving the parameter of Polynomial.

BFormSpline It represents B-form spline $\in \mathbb{S}_n^{n-1}, \forall n$.
Derived from `Spline`.
Implemented by override pure virtual functions of the base class. **Related mathematical theories are introduced below.**
Denote the number of the knots are N, then there are $(N+n-1)$ base functions. They are generated recursively from recursion base `BFSRB`. As the spline is the linear combination of base functions, there are $(N+n-1)$ variables.
Using the values of the spline at the N nodes, we obtain N equations. Using the boundary conditions, we obtain $n-1$ equations.
Write all of the conditions into the linear system, we get A and b. Then generation of the piecewise polynomial is done by doing linear combination of base functions.

BFS1 It represents B-form spline $\in \mathbb{S}_1^0, \forall n$.
Derived form `BFormSpline`.
This is the specified case of BFormSpline, n=1. Thus it is implemented only by call the constructor of the BFormSpline and set the parameter to be 1.
Overload `void generate();`, as the boundary condition is always nothing.

BFS3 It represents B-form spline $\in \mathbb{S}_3^2$.
Derived form `BFormSpline`.
Implemented like BFS1 way, just replacing the parameter from 1 to 3.
It allows 5 kind of boundary conditions (all defined in book[2]), done by override `virtual void addBoundaryCondition(..`

BFS_Thm_3_58 It represents B-form spline defined in Theorem 3.58[4], a spline $\in \mathbb{S}_2^1$, whose knots are not interpolation points.
The generation procedure is same as BFormSpline.
So I only need to override `virtual void addBoundaryCondition(...);`.

CurveFitter It represents a curve fitter, which can fit curve in $\mathbb{R}^2$.
It allows 2 ways of parameterization, using uniform parameter and using cumulative chordal length as parameter.
It is worth mentioning that the matrix A is same in 2 splines, and `void generate_A(...);` allows to reuse generated A by passing in pointer to generated spline, which speed up the calculation.
Related mathematical theories are introduced below.
We have a list of value point on curve. And we can generate the list of independent variable by 2 ways of parameterization above. Combining the independent variable and the value point on a certain direction (x or y), we get 2 list of points on function. Fitting 2 lists of points separately, using `class PFS3`, results in two splines,

which are the estimates of the curve in the x- and y-directions. Since they share the same independent variable, the curve fitting is thus completed.

SphereCurveFitter It represents a sphere curve fitter, which can fit curve on sphere in $\mathbb{R}^3$. **Related mathematical theories are introduced below.**
We take interpolation points on the curve and perform a gnomonic projection to obtain interpolation points in $\mathbb{R}^2$. Then use `class CurveFitter` to get the fitted curve. Finally, the spline is projected back onto the sphere, completing the curve fitting on the spherical surface.

LatexOutputer It is a class that can generate .tex files to plot function, curves, etc.

NameLibrary It is a class that can randomly and non-repeatedly select names from a given name pool. There it is for `LatexOutputer` to get random color.

...Exception In file `Exceptions.hpp`. It defines exception of the project.

EquationSolver It is needed in `class Polynomial` to implement a member function (which is not used in project but used in Chapter2 Hw).

You can get more information about the design of the project from doxygen generated files ./doc/html/index.html.

Public interface

Spline Although `Spline` is abstract class, it offers the public interface which are derived by all of its sub-class. There are 3 public interfaces.

`generate();` The first parameter `const Spline* s` is for CurveFitter. If the matrix is already generated, we can reuse it. If not, you can offer nullptr.
The second parameter `FunctionPointList fList` is the interpolation points, they are points on the function. If you defines a function (don't forget to assign the definition domain), you can use its member function `generatePointList(int number,int derivative_order=0);` to easily get that.
The third parameter `const BoundaryCondition& boundaryCondition=BoundaryCondition_Periodic{}` is the boundary condition. You can objectify a sort of boundary condition class (the sub-class, and offer values it need) and pass it in.
The fourth parameter `const IndependentVariableList& knotList={}` is for spline in Thm 3.58, whose knots are not interpolation points. Then you should assign the knots by this parameter.
As some splines only require a subset of the above parameters, they overload the public interface, e.g. $s \in \mathbb{S}_1^0$ don't need boundary conditions, so this parameter doesn't appear in its parameter list. You can refer to their public interfaces when use.

`print_Latex(); print_Latex_Sole();` The former one is for outputting the spline into a complete .tex files. The latter one is for adding the spline to a unfinished .tex files. They all actually call the function of `class LatexOutputer`, then the outputting is convenient. You can refer to source codes in src/ and doxygen generated files to know more about them.

Specific Spline To complete a spline interpolation, you just need to construct the spline you want to use, and then call functions mentioned above.

References

[1] Qinghai Zhang. "Constructing PP-form splines, Section 3.1.1, Lemma 3.4 and Lemma 3.6". In: *Handouts NumPDEs*. 2024, pp. 20,21.

[2] Qinghai Zhang. "Constructing PP-form splines, Section 3.1.1, Definition 3.5". In: *Handouts NumPDEs*. 2024, p. 21.

[3] Qinghai Zhang. "The local support of B-splines, Section 3.2.2, Definition 3.23". In: *Handouts NumPDEs*. 2024, p. 24.

[4] Qinghai Zhang. "Cardinal B-splines, Section 3.2.7, Theorem 3.58". In: *Handouts NumPDEs*. 2024, p. 29.